Lab 7: Fuzzing for Input Validation Bugs

Name: Diya Agarwal **SRN:** PES1UG24AM096 **Repo:** SETAPESU26/L6_L7_31_coupon_validator (lab7)

Application: `coupon_validator.py` **Tests:** `test_coupon_validator.py` (Hypothesis 6.170, pytest 9.2)

Bugs found by fuzzing

Function	Input found	Buggy behaviour	Fix
split_code	None	AttributeError	Reject non-strings
split_code	'' / 'save20'	ValueError: not enough values to unpack	Full regex match letters-dash-digits
split_code	'SAVE--20'	ValueError: too many values	Same regex
split_code	'-20'	Accepted as ('', 20)	Prefix must be 1+ letters
split_code	'SAVE-+20', Arabic-digit 3	Accepted by int()	ASCII digits only
most_expensive	None	AttributeError	Reject non-strings
most_expensive	'' / '1,,3' / 'abc'	ValueError from float()	Validate every item, return None
most_expensive	'nan', 'inf', '1e400'	Returned nan / inf	Reject non-finite values
most_expensive	'-5'	Negative price accepted	Non-negative numbers only
expiry_days	None	AttributeError	Reject non-strings
expiry_days	'' / '2024--5-01'	ValueError from int()	Regex YYYY-MM-DD
expiry_days	'2024-00-00', '2025-13-45'	Accepted, wrong number	Validate with datetime.date
expiry_days	'2023-12-31'	Returned -4	Reject years before 2024

All three functions now return `None` for input they cannot handle, and `main()` prints a clear message instead of crashing. Valid input gives exactly the same results as before.

Step 1: Install dependencies

```
pip install hypothesis
pip install pytest
```

Step 2: Run the buggy application

Step 2 - buggy app, valid input

```
> python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save-20
Prefix and Value: ('SAVE', 20)

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,49.5,320
Most Expensive Item: 320.0

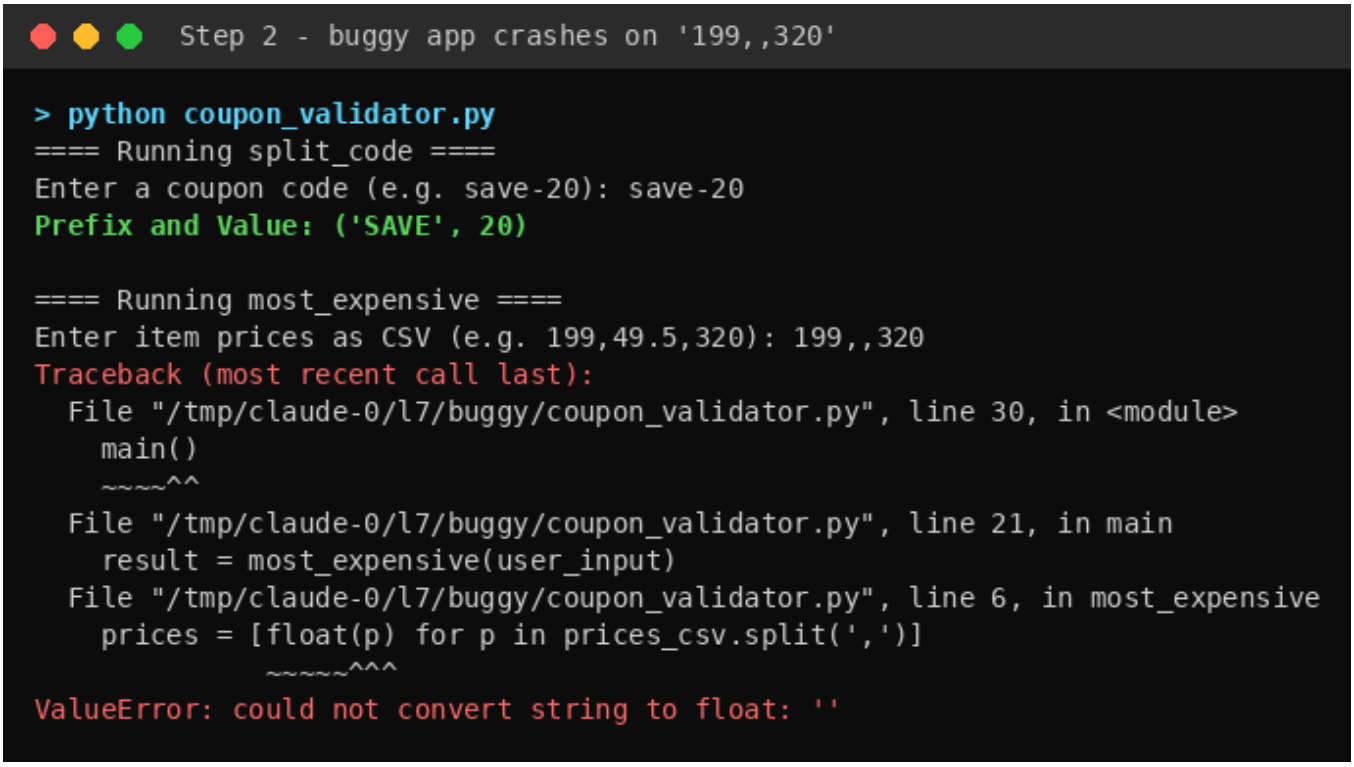
==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-03-15
Days Since 2024: 440
```

Valid input works.

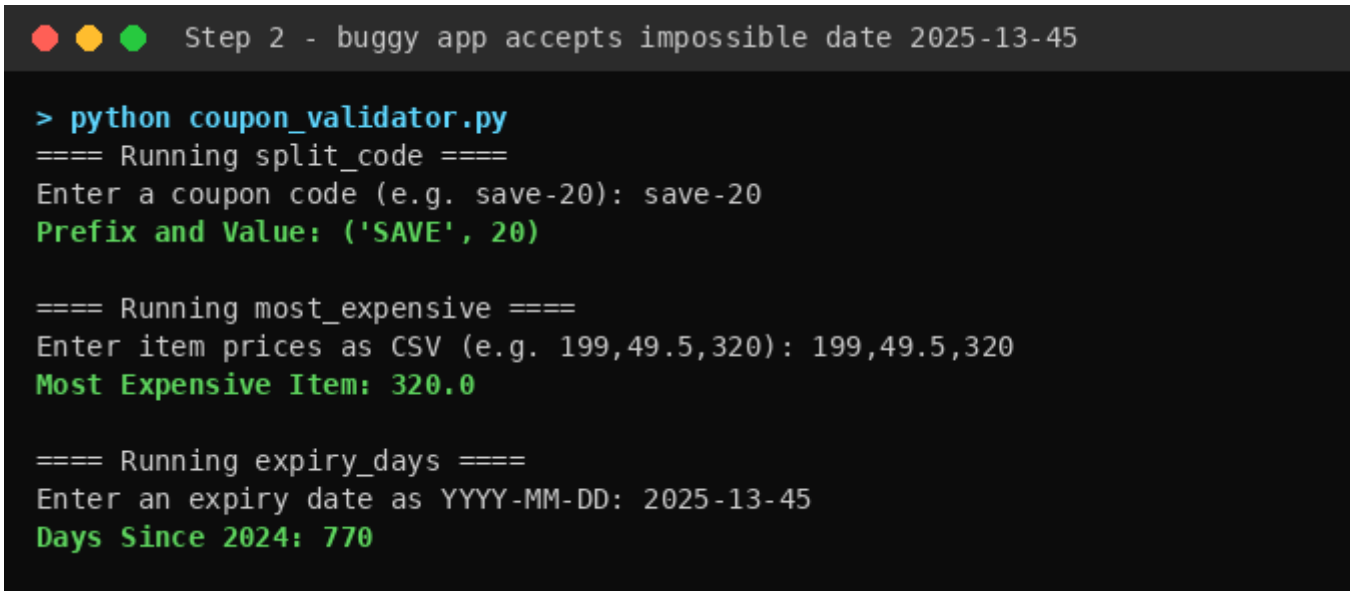
Step 2 - buggy app crashes on 'save20'

```
> python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save20
Traceback (most recent call last):
  File "/tmp/claude-0/l7/buggy/coupon_validator.py", line 30, in <module>
    main()
    ~~~~^^
  File "/tmp/claude-0/l7/buggy/coupon_validator.py", line 16, in main
    result = split_code(user_input)
  File "/tmp/claude-0/l7/buggy/coupon_validator.py", line 2, in split_code
    prefix, value = code.strip().upper().split('-')
    ^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

A code without a dash crashes split_code.



An empty item in the CSV crashes most_expensive.

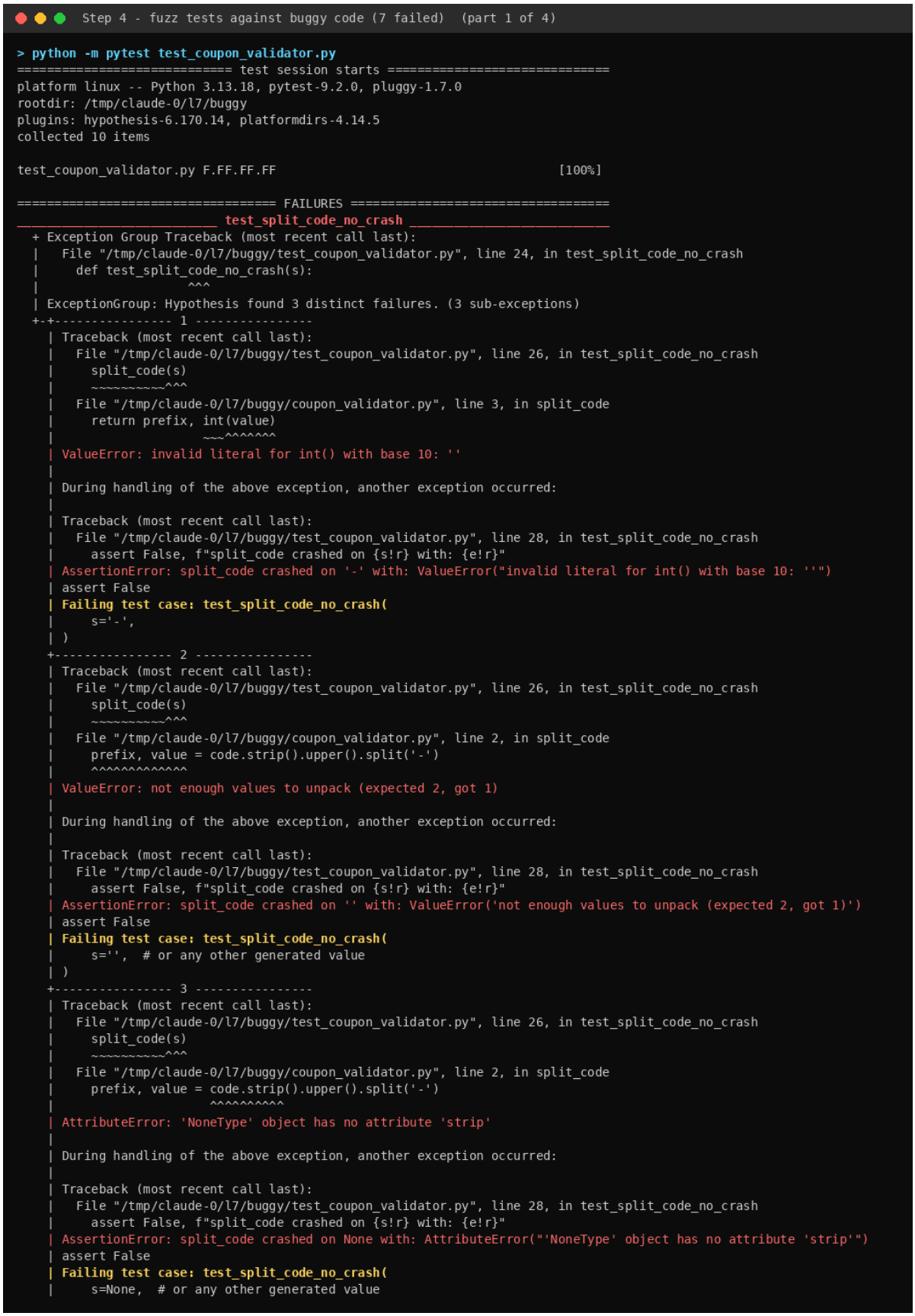


Month 13, day 45 is silently accepted and gives a wrong answer.

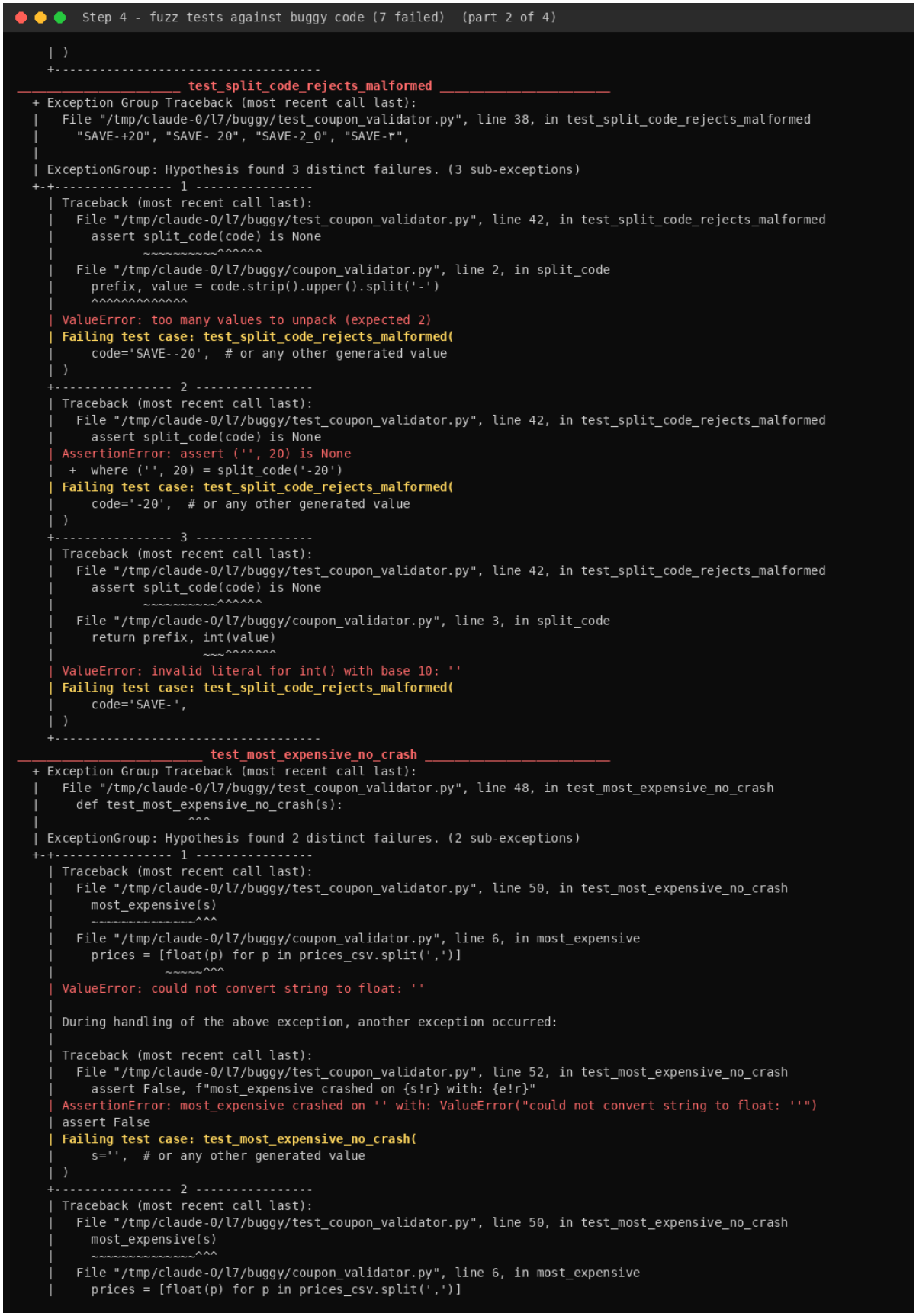
Step 3: Hypothesis tests written

For each function: a **no-crash** fuzz test on `st.text() | st.none()`; a **property** test on valid generated input (round-trip for split_code, equals `max()` for most_expensive, matches the formula for real dates); and a **rejection** test for malformed input that must return None. 10 tests, 300 examples each.

Step 4: Fuzz tests against the buggy code (7 failed, 3 passed)



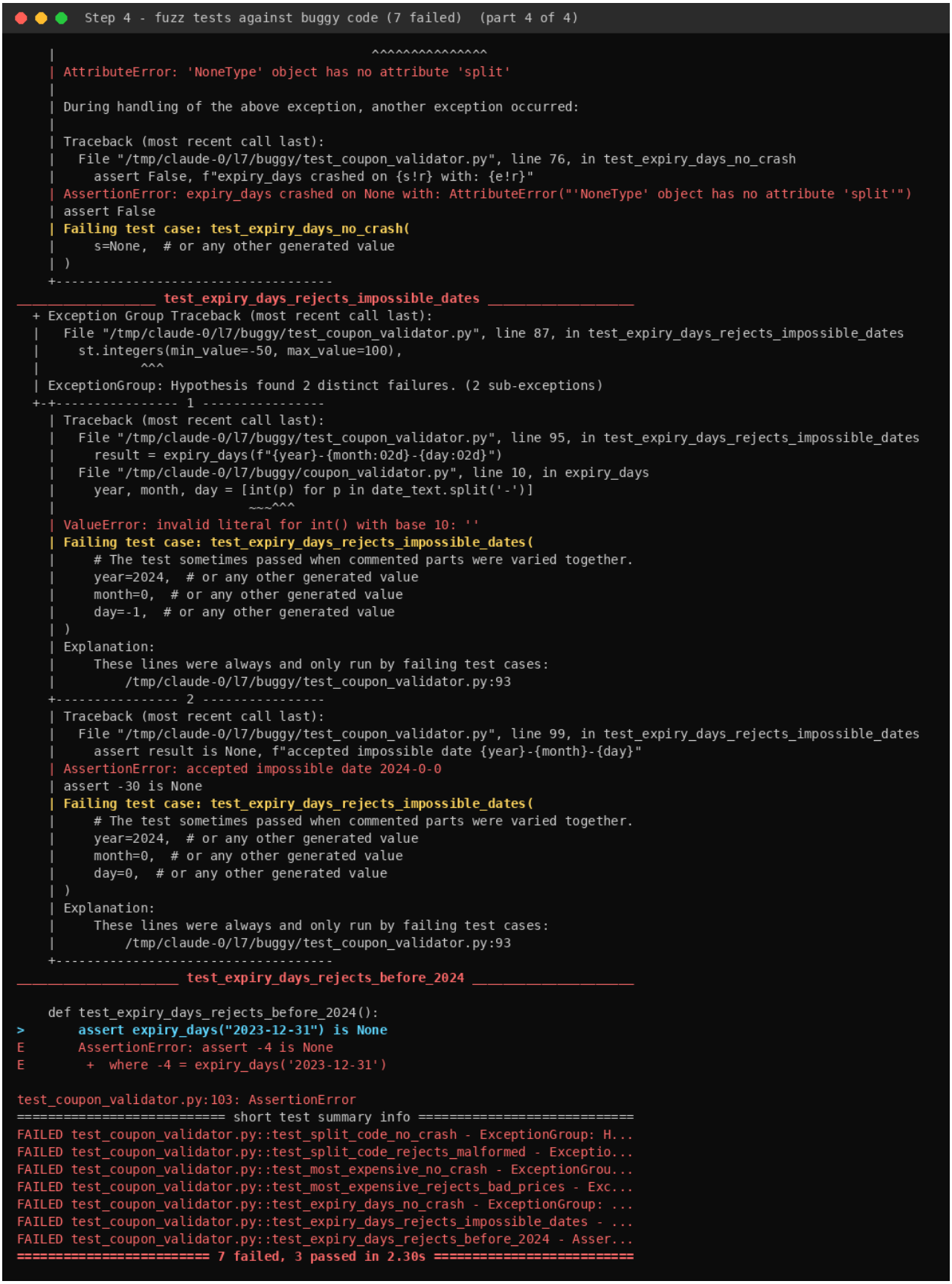
pytest output, part 1 of 4.



pytest output, part 2 of 4.



pytest output, part 3 of 4.



pytest output, part 4 of 4.

Step 5: Run the fixed application

Step 5 - fixed app, valid input

```
> python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save-20
Prefix and Value: ('SAVE', 20)

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,49.5,320
Most Expensive Item: 320.0

==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-03-15
Days Since 2024: 440
```

Valid input: same results as the original.

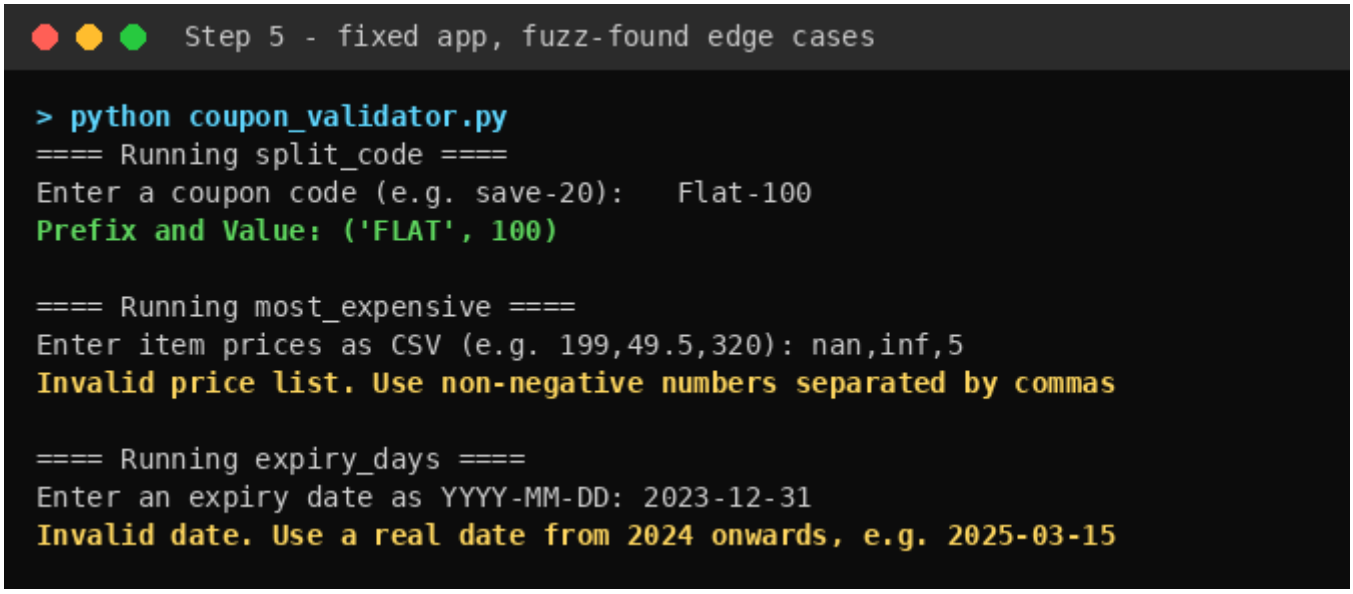
Step 5 - fixed app, invalid input handled

```
> python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save20
Invalid coupon code. Use letters, a dash and a number, e.g. SAVE-20

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,,320
Invalid price list. Use non-negative numbers separated by commas

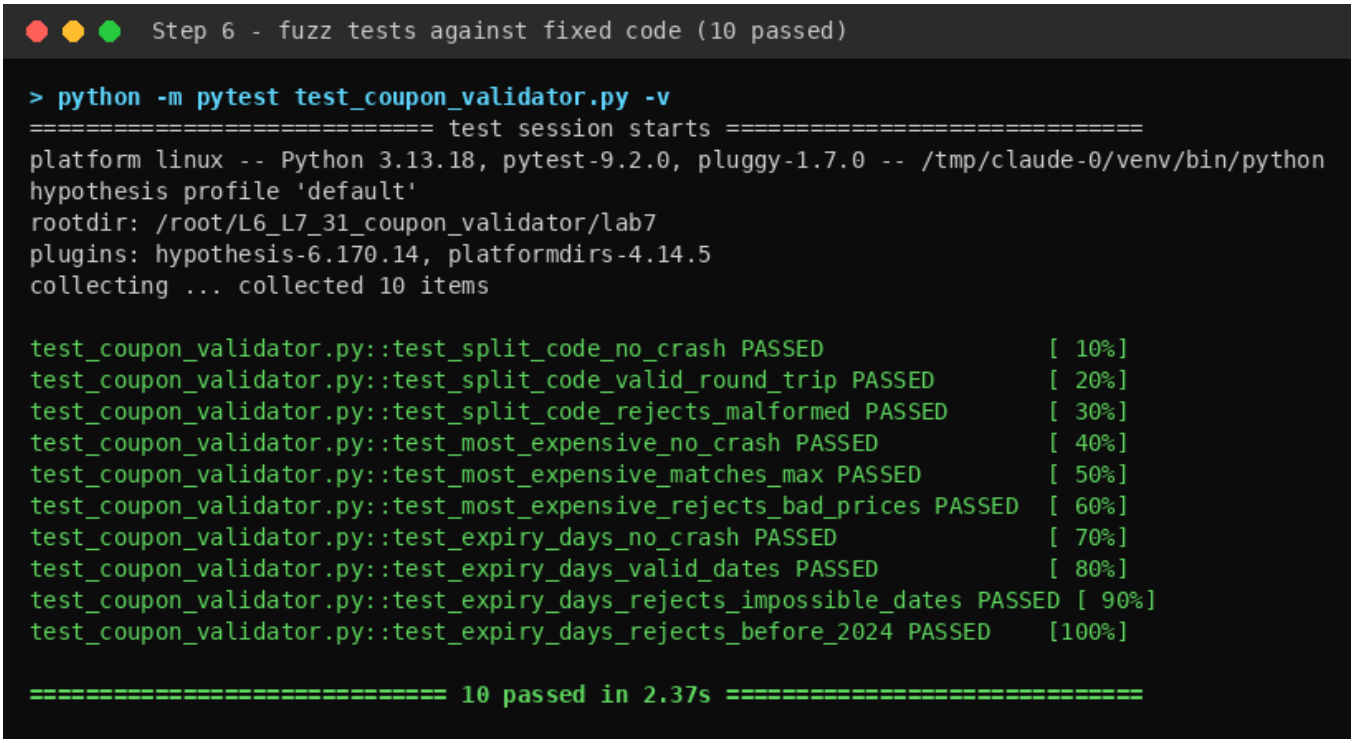
==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-13-45
Invalid date. Use a real date from 2024 onwards, e.g. 2025-03-15
```

The inputs that crashed before are now handled.



Fuzz-found edge cases: whitespace and case accepted; nan/inf and pre-2024 dates rejected.

Step 6: Re-run fuzz tests against the fixed code (10 passed)



All 10 Hypothesis tests pass. (A separate run with 5,000 examples per test also passed.)